```
Output:
int(9223372036854773760)
```

Floats

Floats exist to handle numbers that have fractional components, i.e., numbers with a decimal point such as *1.729*. In addition, if we need to deal with very big numbers, we need floats for those too, and we've just seen how numbers are automatically converted to floats when they exceed PHP_INT_MAX. Why would this be the case? What can a float do that an int can't?

Floating-point numbers (as defined by the IEEE-754 standard) have three components: a sign bit, bits to represent a *mantissa*, and bits for the *exponent*. If you remember the scientific notation from the "Number Systems" section of this chapter, it had two numeric portions separated by an "E" character, the rightmost digits being the power(s) of ten that the leftmost were multiplied by (thus `1.729E3` = $1.729 * 10^3 = 1729$). The mantissa (or *magnitude*) is the left-hand number and the exponent is the right-hand. For a 64-bit floating-point number, 1 bit is used for the sign, 52 bits are used for the mantissa, and the remaining 11 for the exponent.

This means floats can store decimal numbers that have hundreds of powers of ten, far greater in magnitude than the 64-bit *PHP_INT_MAX* (which tops out at eighteen powers of ten), but at the cost of precision: the mantissa can only reliably store up to fifteen significant digits (some sixteen-digit decimal numbers can be represented in 52 bits, but not all of them). Just like integers, floating-point numbers come with their own caveats and limitations:

```
print PHP_FLOAT_MAX . PHP_EOL;
print PHP_FLOAT_MIN . PHP_EOL;
print PHP_FLOAT_DIG . PHP_EOL;
```

```
Output:
1.7976931348623E+308
2.2250738585072E-308
15
```

Logically, of course, there are upper and lower limits on float size, for exactly the same reasons as there are integer limits. The values of PHP_FLOAT_MAX and PHP_FLOAT_MIN depend on the platform running PHP (32-bit vs. 64-bit), just as before. The constant